



PROJECT REPORT

ON

TWEET MANAGEMENT SYSTEM



A TWITTER-STYLE LINKED LIST ENGINE



HEAD → TWEET 1001 → TWEET 1002 → TWEET 1003 → TWEET 1004 → ... → NULL



INSERT
TWEET



DELETE
TWEET



SEARCH
TWEET



TRAVERSE
TIMELINE



REVERSE
LIST



COUNT
NODES



STUDENT NAME : Tejasvi Maheshwari



COURSE : BCA



SEMESTER : 2nd Semester



SUBJECT : C++ Programming & Data Structure



SESSION : 2025-26



COLLEGE : Dezyne École College



Today A Reader,
Tomorrow A Leader

Dezyne École College

www.dezyneecole.com

Title of Project

TWEET MANAGEMENT SYSTEM

Subject Name

Data Structure and C++

Submitted By

Tejasvi Maheshwari

Department - BCA

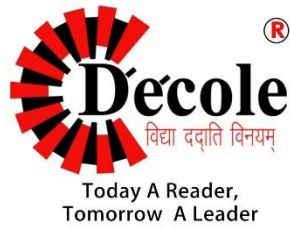
Semester - Second

Session – 2025-26

Project Report of
TWEET MANAGEMENT SYSTEM

Submitted to
Dezyne École College
Towards the Partial Fulfilment of
BCA Second Semester

By
Tejasvi Maheshwari



www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,
Tejasvi Maheshwari
Department of BCA
Dezyne École College, Ajmer

Contents

1. Introduction.....	1
2. Theoretical Foundation	2
2.1 Why Linked Lists Matter.....	2
2.2 What is a Linked List?.....	2
2.3 Linked List vs Array	2
2.4 Types of Linked Lists	2
3. Data Structure Design	3
3.1 The NODE Class (Singly)	3
3.2 The DNODE Class (Doubly).....	3
3.3 List Manager Classes	3
4. Pointer Mechanics — A Deep Dive.....	4
5. Singly Linked List Walkthrough.....	5
5.1 Insertion Operations.....	5
5.1.1 Insert at Beginning.....	5
5.1.2 Insert at End.....	5
5.1.3 Insert at Given Position (e.g. position 3).....	6
5.2 Traversal & Search	6
5.2.1 Forward Traversal	6
5.2.2 Search by Tweet ID / User Name.....	7
5.3 Deletion Operations	7
5.3.1 Delete from Beginning	7
5.3.2 Delete from End.....	8
5.3.3 Delete at Given Position (e.g. position 2).....	8
6. Doubly Linked List Walkthrough	10
6.1 Insertion Operations.....	10
6.1.1 Insert at Beginning.....	10
6.1.2 Insert at End.....	10
6.1.3 Insert at Given Position (e.g. position 3).....	11
6.2 Forward & Backward Traversal.....	11
6.2.1 Forward (START → NEXT)	12
6.2.2 Backward (END → PREV).....	12
Backward: tail -> prev ... NULL.....	12

6.3 Search Operation.....	12
6.4 Deletion Operations	13
6.4.1 Delete from Beginning	13
6.4.2 Delete from End	13
6.4.3 Delete at Given Position (e.g. position 2)	14
7. Appendix — Source Code.....	15
7.1 main.cpp	15
Main File Menu	16
7.2 singlyLinkedList.hpp	16
SINGLY LINKED LIST OUTPUT UI	28
Singly Main Menu.....	28
► Insertion Menu.....	28
1. Insert at Beginning	28
2. Insert at End.....	29
3. Insert at Position	29
► Deletion Menu	29
1. Delete from Beginning	30
2. Delete from End	30
3. Delete from Position.....	30
► Searching Menu	31
1. Search Operation	31
► Traversal Menu	31
1. Traversal	31
► Count Menu.....	32
7.3 DoublyLinkedList.hpp.....	33
DOUBLY LINKED LIST OUTPUT	45
►Doubly Main Menu.....	45
► Insertion Menu.....	45
1. Insert at Beginning	45
2. Insert at End.....	46
3. Insert at Position	46
► Deletion Menu	46
1. Delete from Beginning	46
2. Delete from End	47

3. Delete from Position.....	47
► Searching Menu	47
1. Search Operation.....	48
► Traversal Menu	48
1. Forward & Backward Traversal	48
► Count Menu	49

1. Introduction

The Tweet & Data Management System stores tweet records in dynamic memory using two flavours of linked lists. This document is a visual map of how each operation moves bytes on the heap .

Aspect	Details
Project	Tweet & Data Management System
Language	C++ (procedural / class-based)
Data Unit	Tweet → { ID, USERNAME, TWEET }
Structures	Singly Linked List, Doubly Linked List
Operations	Insert, Traverse, Search, Delete, Count

Real-World Context — Twitter Timeline

Twitter's home timeline shows tweets in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new tweet always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any tweet immediately — $O(1)$.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs define the **Linked List** — a data structure that does all three efficiently.

2. Theoretical Foundation

2.1 Why Linked Lists Matter

A tweet feed grows and shrinks unpredictably. Linked lists allocate one node at a time on the heap, so memory cost scales exactly with the number of tweets — no resizing, no shifting.

2.2 What is a Linked List?

A chain of self-referential nodes. Each node carries data plus the address of the next (and, in a doubly list, the previous) node. The chain is anchored by a HEAD pointer; the tail points to NULL.

2.3 Linked List vs Array

Criterion	Array	Linked List
Memory layout	Contiguous block	Scattered nodes on the heap
Size	Fixed at allocation	Grows/shrinks at runtime
Insert / Delete (middle)	$O(n)$ — shift elements	$O(1)$ — re-wire pointers
Random access	$O(1)$ by index	$O(n)$ — must traverse
Memory overhead	None per element	Pointer field per node
Cache locality	Excellent	Poor

2.4 Types of Linked Lists

Type	Pointer Fields	Direction
Singly Linked	NEXT only	Forward only
Doubly Linked	PREV + NEXT	Forward & Backward
Circular Singly	NEXT (tail → head)	Forward, looped
Circular Doubly	PREV + NEXT (looped)	Both, looped

3. Data Structure Design

3.1 The NODE Class (Singly)

```
class NODE {
public:
    string ID;           // e.g. "TWEET1001"
    string NAME;         // user name
    string TWEET;        // tweet body
    NODE *NEXT;          // address of next node

    NODE(string X, string Y, string Z) {
        ID = X; NAME = Y; TWEET = Z;
        NEXT = nullptr;
    }
};
```

Memory footprint of one NODE (illustrative):

Field	Type	Role
ID	string	Auto-generated tweet identifier
NAME	string	Author handle
TWEET	string	Tweet payload
NEXT	NODE*	Address of the next node, or nullptr

3.2 The DNODE Class (Doubly)

```
class DNODE {
public:
    DNODE *PREV;         // address of previous node
    string ID;
    string UNAME;
    string TWEET;
    DNODE *NEXT;         // address of next node

    DNODE(string X, string Y, string Z) {
        PREV = nullptr;
        ID = X; UNAME = Y; TWEET = Z;
        NEXT = nullptr;
    }
};
```

3.3 List Manager Classes

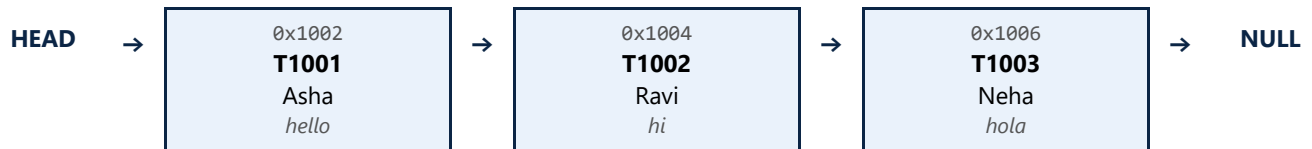
Class	Owns	Tracks
singlylinklist	NODE* START, *END	CountNode, AutoTID
Doublylinklist	DNODE* START, *END	CountNode, AutoTID

4. Pointer Mechanics — A Deep Dive

A pointer is a variable whose value is the address of another variable. Drawing pointers as arrows hides their true nature: they are integers that index the heap.

Symbol	Means	Example
NODE *p	Declare p as address of a NODE	NODE *p;
new NODE(...)	Allocate node on heap, return its address	p = new NODE(...);
p->NEXT	Field NEXT of the node p points to	p->NEXT = q;
delete p	Free the heap block at address p	delete p;
nullptr	The address "nowhere" (end-of-list)	p->NEXT = nullptr;

Below: the same chain seen as the programmer's mental model (left) and as raw memory addresses (right).



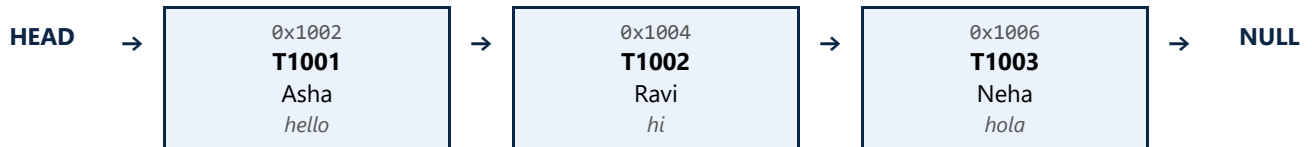
5. Singly Linked List Walkthrough

5.1 Insertion Operations

5.1.1 Insert at Beginning

A new node is created and linked at the head. The previous head becomes the second node.

STATE (PRE) — Memory Heap

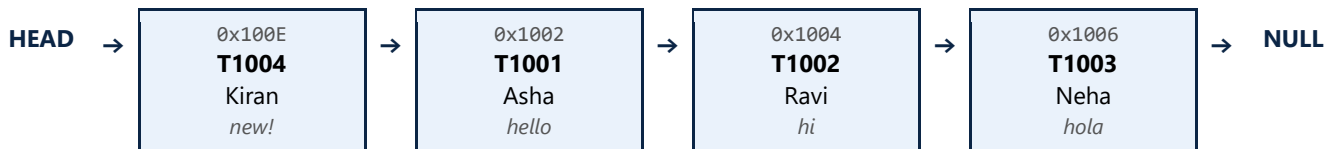


ACTION (CODE) — Pointer Reassignment

```

NODE *TEMP = new NODE(TID, UNAME, UTWEET);
if (START == nullptr) {
    END = START = TEMP;
} else {
    TEMP->NEXT = START;    // new node points to old head
    START      = TEMP;    // head now refers to new node
}
CountNode++; AutoTID++;
  
```

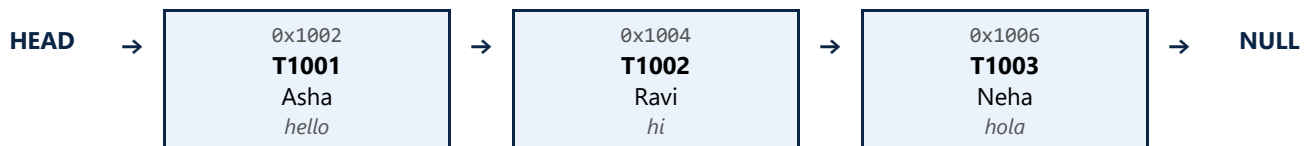
RESULT (POST) — Healed Links



5.1.2 Insert at End

Traverse to the last node, set its next pointer to the new node.

STATE (PRE) — Memory Heap

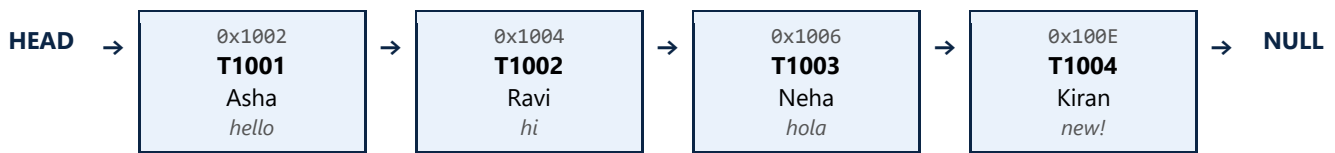


ACTION (CODE) — Pointer Reassignment

```

NODE *TEMP = new NODE(TID, UNAME, UTWEET);
if (START == nullptr) {
    END = START = TEMP;
} else {
    END->NEXT = TEMP;    // current tail links to new node
    END      = TEMP;    // tail pointer advances
}
  
```

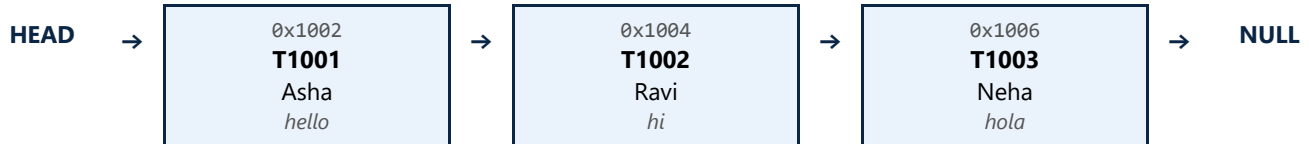
RESULT (POST) — Healed Links



5.1.3 Insert at Given Position (e.g. position 3)

Traverse to position-3, splice the new node between it and the next node.

STATE IPRI — Memory Heap

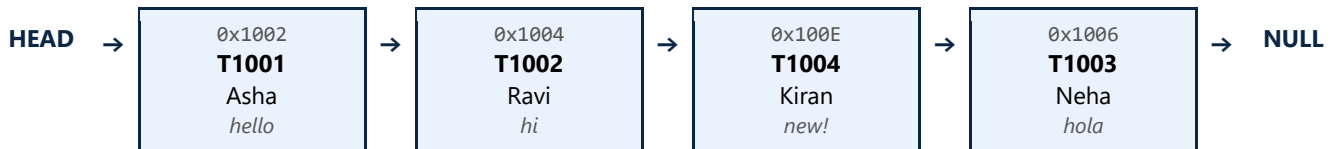


ACTION ICODEI — Pointer Reassignment

```

NODE *TEMP1 = START;
for (int i = 2; i < pos; i++) TEMP1 = TEMP1->NEXT;
NODE *TEMP = new NODE(TID, UNAME, UTWEET);
TEMP->NEXT = TEMP1->NEXT; // new node inherits the rest
TEMP1->NEXT = TEMP;      // predecessor adopts new node
  
```

RESULT IPOSTI — Healed Links



5.2 Traversal & Search

5.2.1 Forward Traversal

Walk from head to NULL printing each node.

```

NODE *TEMP = START;
while (TEMP != nullptr) {
    cout << "[ " << TEMP->ID << ", "
        << TEMP->NAME << ", " << TEMP->TWEET << " ] -> ";
    TEMP = TEMP->NEXT;
}
cout << "Null";
  
```

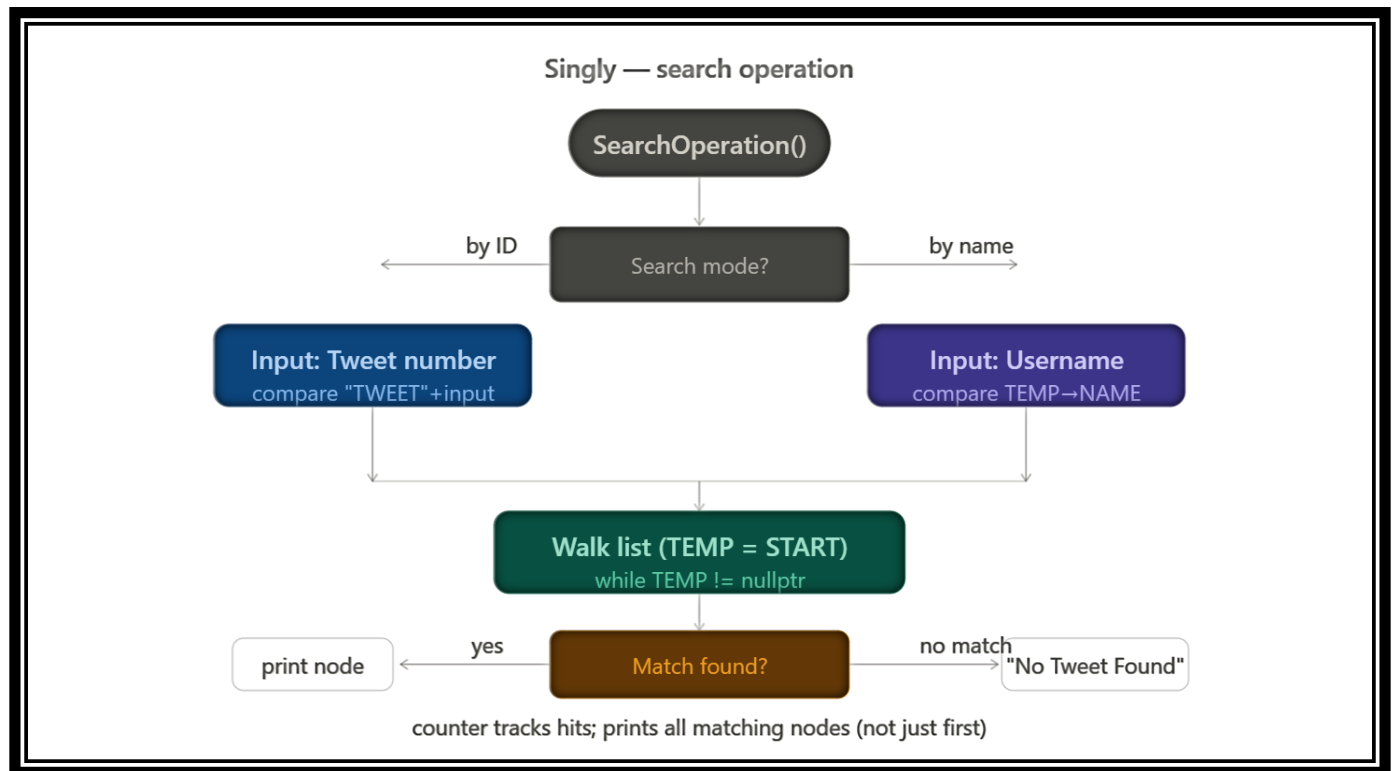
Sample run:

```
-----Link List-----
```

```
Head -> [ TWEET1001 , Asha , hello ] -> [ TWEET1002 , Ravi , hi ] -> [ TWEET1003 , Neha , hola ] -> Null
```

5.2.2 Search by Tweet ID / User Name

Linear search by Tweet ID or User Name; print matching node.



```

while (TEMP != nullptr) {
    if (TEMP->ID == ("TWEET" + TweetID)) {
        cout << "[ " << TEMP->ID << " , "
            << TEMP->NAME << " , " << TEMP->TWEET << " ]";
        counter++;
    }
    TEMP = TEMP->NEXT;
}
if (counter == 0) cout << "No Tweet Found";

```

Search Tweet by Tweet ID-----

Enter Tweet ID to find the Tweet -- 1002

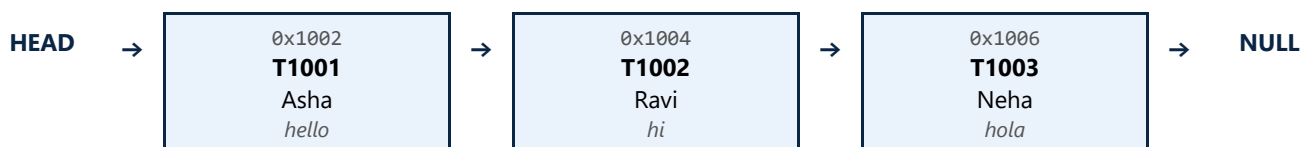
[TWEET1002 , Ravi , hi]

5.3 Deletion Operations

5.3.1 Delete from Beginning

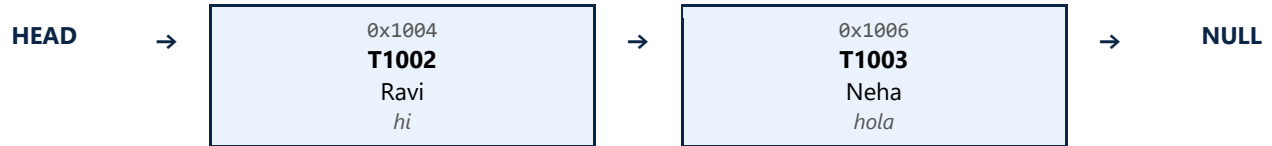
Move head to head->next, free the old head node.

STATE IPREI — Memory Heap

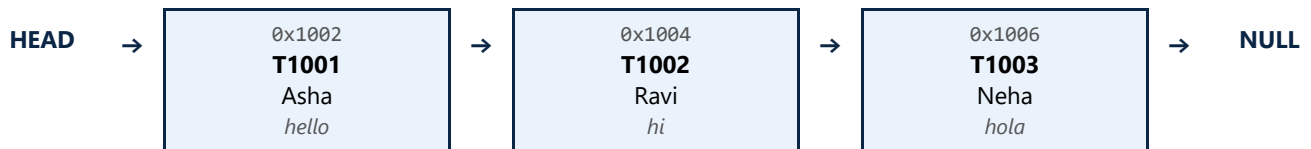


ACTION ICODEI — Pointer Reassignment

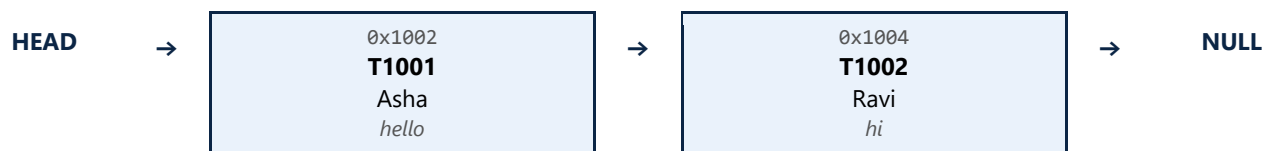
```
TEMP = START;
START = TEMP->NEXT; // head jumps over the old node
delete TEMP;        // heap block freed
CountNode--;
```

RESULT IPOSTI — Healed Links**5.3.2 Delete from End**

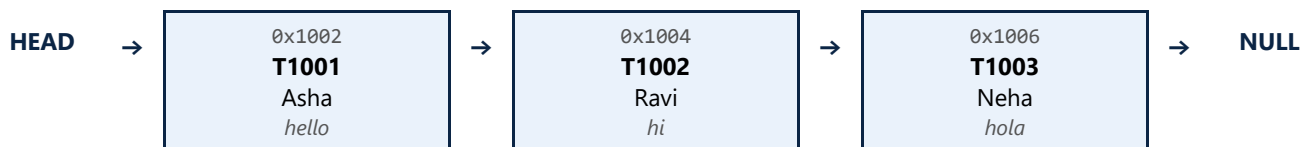
Walk to second-last node, set its next to NULL, free last node.

STATE IPREI — Memory Heap**ACTION ICODEI — Pointer Reassignment**

```
TEMP = START;
while (TEMP->NEXT != END) TEMP = TEMP->NEXT; // walk to penultimate
NODE *TEMP1 = END;
TEMP->NEXT = nullptr; // new tail terminates
END = TEMP;
delete TEMP1;
```

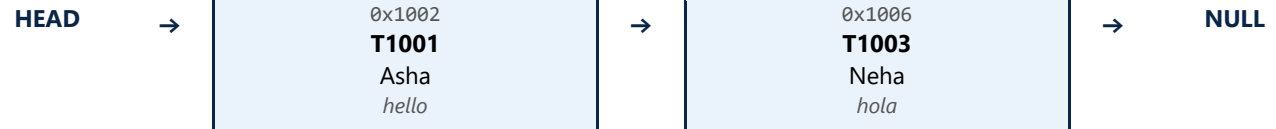
RESULT IPOSTI — Healed Links**5.3.3 Delete at Given Position (e.g. position 2)**

Walk to position-2, bypass the target node, free it.

STATE IPREI — Memory Heap**ACTION ICODEI — Pointer Reassignment**

```
TEMP = START;
```

```
for (int i = 2; i < pos; i++) TEMP = TEMP->NEXT;  
NODE *TEMP1 = TEMP->NEXT;    // node to remove  
TEMP->NEXT = TEMP1->NEXT;    // bypass it  
delete TEMP1;
```

RESULT (POST) — Healed Links

6. Doubly Linked List Walkthrough

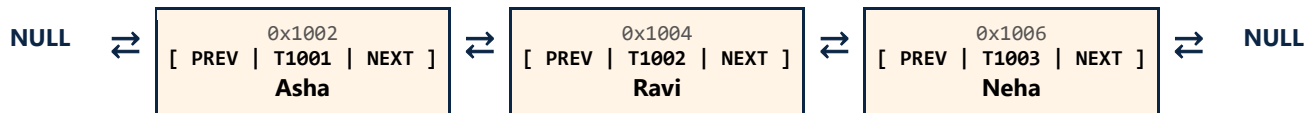
Each DNODE keeps two pointers, so every edit must mend both directions. The diagrams below use the [PREV | DATA | NEXT] motif to make link surgery obvious.

6.1 Insertion Operations

6.1.1 Insert at Beginning

New node becomes head; old head's prev points to it.

STATE IPREI — Memory Heap

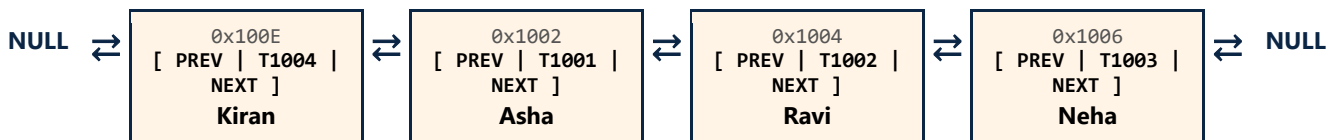


ACTION ICODEI — Pointer Reassignment

```

DNODE *TEMP = new DNODE(TID, UNAME, UTWEET);
if (START == nullptr) {
    END = START = TEMP;
} else {
    TEMP->NEXT = START;    // new -> old head
    START->PREV = TEMP;    // old head -> new
    START      = TEMP;      // head moves
}
  
```

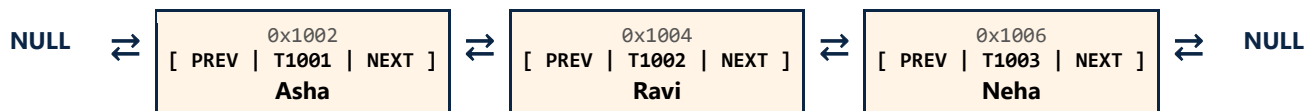
RESULT IPOSTI — Healed Links



6.1.2 Insert at End

Walk to last; set last->next = new, new->prev = last.

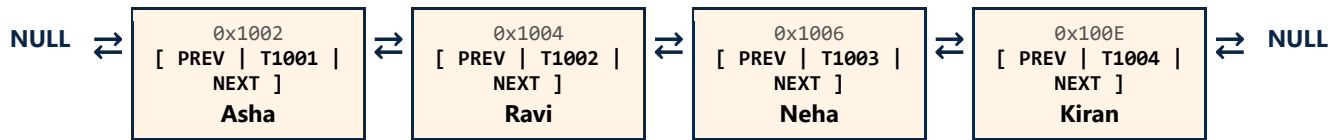
STATE IPREI — Memory Heap



ACTION ICODEI — Pointer Reassignment

```

DNODE *TEMP = new DNODE(TID, UNAME, UTWEET);
if (START == nullptr) {
    END = START = TEMP;
} else {
    END->NEXT = TEMP;    // old tail -> new
    TEMP->PREV = END;    // new -> old tail
    END      = TEMP;      // tail moves
}
  
```

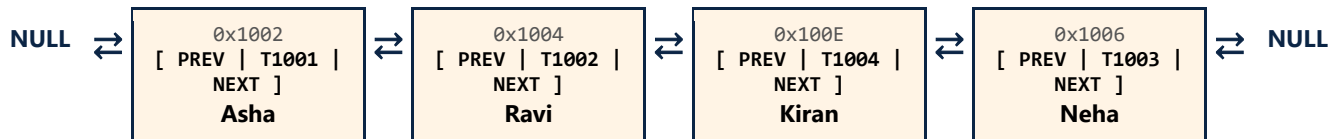
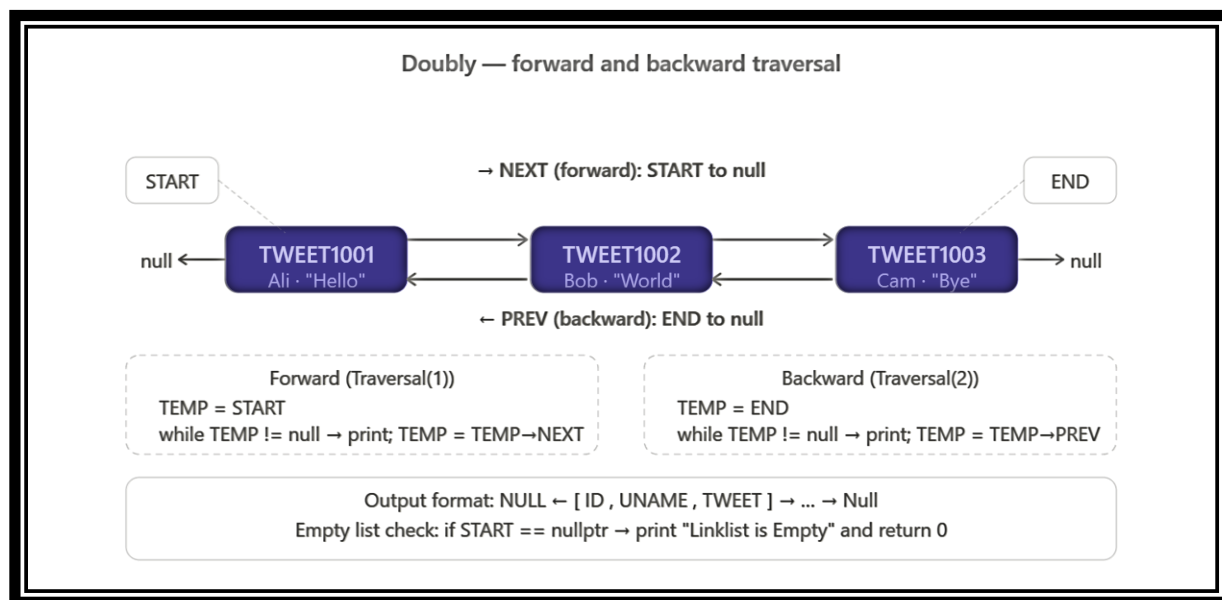
RESULT (POST) — Healed Links**6.1.3 Insert at Given Position (e.g. position 3)**

Re-wire four pointers: prev->new and new->next.

STATE (PRE) — Memory Heap**ACTION (CODE) — Pointer Reassignment**

```

DNODE *TEMP1 = START;
for (int i = 2; i < pos; i++) TEMP1 = TEMP1->NEXT;
DNODE *TEMP = new DNODE(TID, UNAME, UTWEET);
TEMP ->NEXT = TEMP1->NEXT;
TEMP1->NEXT = TEMP;
TEMP ->PREV = TEMP1;
TEMP ->NEXT->PREV = TEMP;  // mend the back-link of successor
  
```

RESULT (POST) — Healed Links**6.2 Forward & Backward Traversal**

6.2.1 Forward (START → NEXT)

Forward: head -> next ... NULL.

```

DNode *TEMP = START;
while (TEMP != nullptr) {
    cout << "[ " << TEMP->ID << ", "
        << TEMP->UNAME << ", " << TEMP->TWEET << " ] -> ";
    TEMP = TEMP->NEXT;
}

```

```

NULL <- [ TWEET1001 , Asha , hello ] -> [ TWEET1002 , Ravi , hi ] -> [ TWEET1003 , Neha ,
hola ] -> Null

```

6.2.2 Backward (END → PREV)

Backward: tail -> prev ... NULL.

```

DNode *TEMP = END;
while (TEMP != nullptr) {
    cout << "[ " << TEMP->ID << ", "
        << TEMP->UNAME << ", " << TEMP->TWEET << " ] -> ";
    TEMP = TEMP->PREV;
}

```

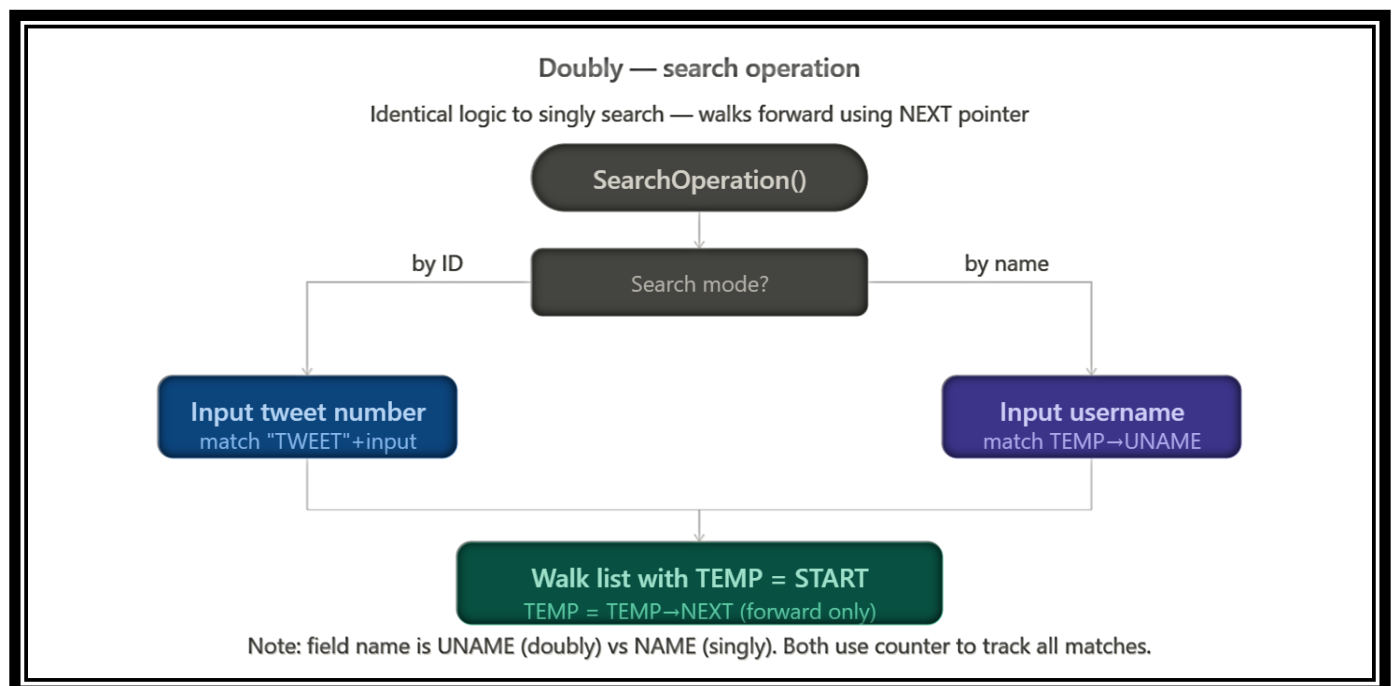
```

NULL <- [ TWEET1003 , Neha , hola ] -> [ TWEET1002 , Ravi , hi ] -> [ TWEET1001 , Asha ,
hello ] -> Null

```

6.3 Search Operation

Search walks forward from START, comparing against TweetID or username. Identical to the singly version, but operates on DNode.



```

while (TEMP != nullptr) {
    if (TEMP->UNAME == uname) {
        cout << "[ " << TEMP->ID << ", "
            << TEMP->UNAME << ", " << TEMP->TWEET << " ]";
        counter++;
    }
    TEMP = TEMP->NEXT;
}

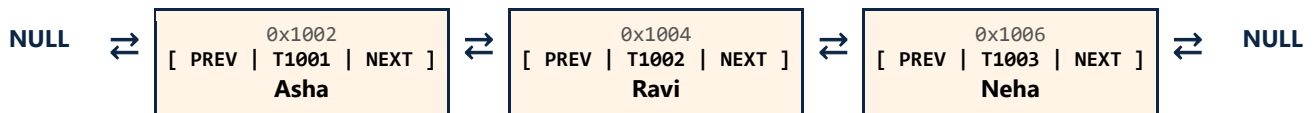
```

6.4 Deletion Operations

6.4.1 Delete from Beginning

head = head->next; head->prev = NULL; free old head.

STATE IPREI — Memory Heap



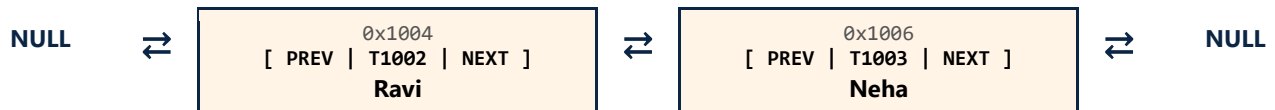
ACTION ICODEI — Pointer Reassignment

```

TEMP = START;
START = TEMP->NEXT;
START->PREV = nullptr;    // new head has no predecessor
delete TEMP;

```

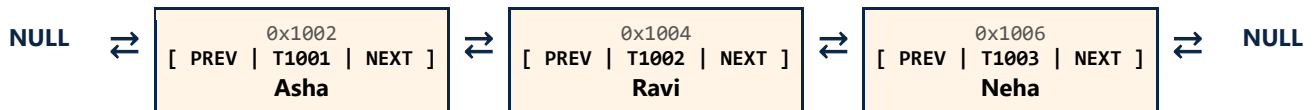
RESULT IPOSTI — Healed Links



6.4.2 Delete from End

STATE IPREI — Memory Heap

Walk to last; last->prev->next = NULL; free last.



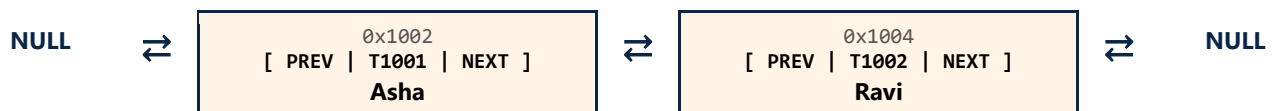
ACTION ICODEI — Pointer Reassignment

```

TEMP = END;
END = END->PREV;    // tail steps back
END->NEXT = nullptr; // terminate the chain
delete TEMP;

```

RESULT IPOSTI — Healed Links



6.4.3 Delete at Given Position (e.g. position 2)

STATE (PRE) — Memory Heap

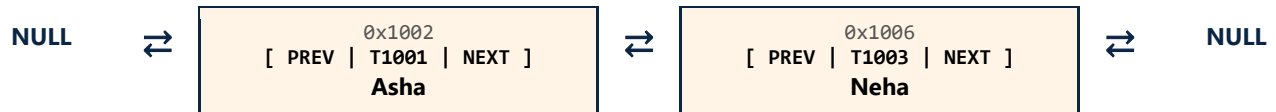
Re-wire prev->next and next->prev around the target, free target.



ACTION (CODE) — Pointer Reassignment

```
TEMP = START;  
for (int i = 1; i < pos; i++) TEMP = TEMP->NEXT;  
TEMP->PREV->NEXT = TEMP->NEXT;    // bridge forward link  
TEMP->NEXT->PREV = TEMP->PREV;    // bridge backward link  
delete TEMP;
```

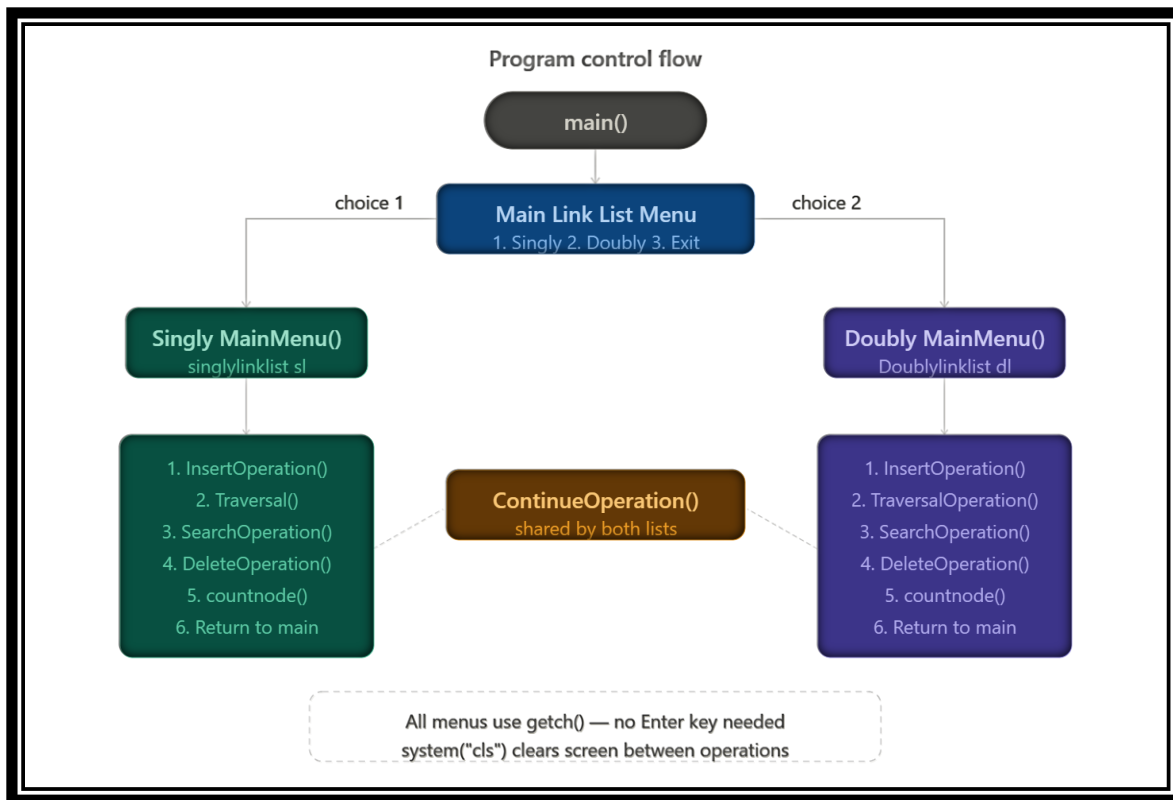
RESULT (POST) — Healed Links



7. Appendix — Source Code

7.1 main.cpp

```
#include "singlyLinkedList.hpp"
#include "DoublyLinkedList.hpp"
int main()
{
    singlylinklist sl;
    Doublylinklist dl;
    while (true)
    {
        system("cls");
        cout<<"\n\n=====Main Link List Menu=====\\n";
        cout<<"1.Singly Link List\\n";
        cout<<"2.Doubly Link List\\n";
        cout<<"3.Exit\\n";
        char c = getch();
        if(c == '1'){
            sl.MainMenu();
        }else if (c == '2'){
            dl.MainMenu();
        }else if (c == '3'){
            break;
        }
    }
    cout<<"\\n\\nThank you for using my Application...\\n\\n";
    return 0;
}
```



Main File Menu

```

-----Main Link List Menu-----
1.Singly Link List
2.Doubly Link List
3.Exit
  
```

7.2 singlyLinkList.hpp

```

#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODE
{
public:
    string ID;
    string NAME;
    string TWEET;
    NODE *NEXT ;

    NODE(string X , string Y , string Z)
    {
  
```

```
ID = X;
NAME = Y;
TWEET = Z;
NEXT = nullptr;
}
};
class singlylinklist
{
public:
    string TID;
    string UNAME;
    string UTWEET;
    int CountNode;
    int AutoTID;
    NODE *START , *END ;
    singlylinklist()
    {
        AutoTID=1001;
        CountNode=0;
        START=nullptr;
        END=nullptr;
    }

    //Get Data
    void getdata()
    {
        TID = "TWEET" + to_string(AutoTID);
        cout<<"\nEnter TID :-"<<TID<<endl;
        cout<<"Enter User Name :- ";
        getline(cin,UNAME);
        cout<<"Enter Tweet :- ";
        getline(cin,UTWEET);
    }

    //Insert Operation
    void InsertOperation()
    {
        system("cls");
        cout<<"\n=====Insertion Opertaion=====\\n";
        cout<<"1.Insert at Beggining\\n";
        cout<<"2.Insert at End\\n";
        cout<<"3.Insert at given Position\\n";
        cout<<"4.Go to Main Menu\\n";
        cout<<"5.Exit\\n";
    }
};
```



```
char c = getch();
if(c == '1'){
    InsertNode(1);
}else if (c == '2'){
    InsertNode(2);
}else if (c == '3'){
    InsertNode(3);
}else if (c == '4'){
    MainMenu();
}else if (c == '5'){
    thankyouLine();
    exit(0);
}
}

//Insert Module
int InsertNode(int x)
{
    system("cls");
    if(x == 1){
        cout<<"-----Insertion at Beggining-----\n";
        getdata();
        NODE *TEMP = new NODE(TID,UNAME,UTWEET);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory\n";
            return 0;
        }
        if(START == nullptr){
            END = START = TEMP;
        }else {
            TEMP->NEXT = START;
            START = TEMP;
        }
        CountNode++;
        AutoTID++;
    }else if (x == 2){
        cout<<"-----Insertion at End-----\n";
        getdata();
        NODE *TEMP = new NODE(TID,UNAME,UTWEET);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory\n";
            return 0;
        }
        if(START == nullptr){
            END = START = TEMP;
```

```
    }else {
        END->NEXT = TEMP;
        END = TEMP;
    }
    CountNode++;
    AutoTID++;
}
else if (x == 3){
    cout<<"-----Insertion at given Position-----\n";
    cout<<"Total Number of Nodes Are -- "<<CountNode<<endl;
    int pos;
    cout<<"Enter Position Between 0 and "<<CountNode<<" - ";
    cin>>pos;
    cin.ignore();
    getdata();
    NODE *TEMP = new NODE(TID,UNAME,UTWEET);
    if (TEMP == nullptr){
        cout<<"Insufficient Memory";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for(i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if(START == nullptr){
        END = START = TEMP;
    }
    else {
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT=TEMP;
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    CountNode++;
    AutoTID++;
}
char c = ContinueOperation();
if(c == '1'){
    InsertNode(x);
}
else if(c == '2'){
    InsertOperation();
}
else if(c == '3'){
    MainMenu();
}
```

```
}

//Show Module
void Traversal()
{
    system("cls");
    cout<<"\n-----Link List-----\n\n";
    cout<<"Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"NULL \n\nLinklist is Empty";
    }
    else {
        while(TEMP != nullptr){
            cout<<"[ "<<TEMP->ID<<" , ";
            cout<<TEMP->NAME<<" , ";
            cout<<TEMP->TWEET<<" ] -> ";
            TEMP=TEMP->NEXT;
        }
        cout<<"Null";
    }
    cout<<"\n\nDo you want to -- \n";
    cout<<"1.Continue the Same\n";
    cout<<"2.Go to Main Menu\n";
    cout<<"3.Exit\n";
    char c = getch();
    if(c == '1'){
        Traversal();
    }else if(c == '2'){
        MainMenu();
    }else if(c == '3'){
        thankyouLine();
        exit(0);
    }
}

//Search Operation
void SearchOperation()
{
    system("cls");
    cout<<"=====Search Module===== \n\n";
    cout<<"1.Search by Tweet ID \n";
    cout<<"2.Search by User Name \n";
    cout<<"3.Main Menu \n";
    cout<<"4.Exit\n";
}
```

```

char c = getch();
if(c == '1'){
    searchNode(1);
}else if(c == '2'){
    searchNode(2);
}else if(c == '3'){
    MainMenu();
}else if(c == '4'){
    exit(0);
}
}

//Search Module
int searchNode(int x)
{
    system("cls");
    if (x == 1){
        cout<<"Search Tweet by Tweet ID-----\n\n";
        NODE *TEMP = START;
        if (START == nullptr){
            cout<<"Head -> Null \n\nLink List is Empty...";
            return 0;
        }else{
            int counter=0;
            cout<<"Enter Tweet ID to find the Tweet -- ";
            string TweetID;
            getline(cin,TweetID);
            while (TEMP != nullptr){
                if(TEMP->ID == ("TWEET" + TweetID)){
                    cout<<"\n[ "<<TEMP->ID<<" , ";
                    cout<<TEMP->NAME<<" , ";
                    cout<<TEMP->TWEET<<" ]\n";
                    counter++;
                }
                TEMP = TEMP->NEXT;
            }
            if(counter == 0){
                cout<<"\nNo Tweet Found\n\n";
            }
        }
    }
    else if(x == 2){
        cout<<"-----Search Tweet by User Name-----\n\n";

        NODE *TEMP = START;
    }
}

```

```

    if (START == nullptr){
        cout<<"Head -> Null \n\nLink List is Empty...";
        return 0;
    }else{
        int counter=0 ;
        cout<<"Enter user Name to find the Tweet -- ";
        string uname;
        getline(cin,uname);
        cout<<endl<<"Head -> ";
        while (TEMP != nullptr){
            if(TEMP->NAME == uname){
                cout<<"\n[ "<<TEMP->ID<<" , ";
                cout<<TEMP->NAME<<" , ";
                cout<<TEMP->TWEET<<" ]\n";
                counter++;
            }
            TEMP = TEMP->NEXT;
        }
        if(counter == 0){
            cout<<"\nNo Tweet Found\n\n";
        }
    }
}
cout<<"Do you want to -- \n";
cout<<"1.Continue the Same\n";
cout<<"2.Go to Main Menu\n";
cout<<"3.Exit\n";
char c = getch();
if(c == '1'){
    searchNode(x);
}else if(c == '2'){
    MainMenu();
}else if(c == '3'){
    thankyouLine();
    exit(0);
}
}

//Delete Operation
void DeleteOperation()
{
    system("cls");
    cout<<"\n=====Deletion Opertaion===== \n";
    cout<<"1.Delete from Beggining\n";
    cout<<"2.Delete from End\n";
}

```

```
cout<<"3.delete from a given Position\n";
cout<<"4.Go to Main Menu\n";
cout<<"5.Exit\n";
char c = getch();
if(c == '1'){
    DeleteNode(1);
}else if (c == '2'){
    DeleteNode(2);
}else if (c == '3'){
    DeleteNode(3);
}else if (c == '4'){
    MainMenu();
}else if (c == '5'){
    thankyouLine();
    exit(0);
}
}

//Delete Module
int DeleteNode(int x)
{
    system("cls");
    NODE *TEMP ;
    if(x == 1){
        cout<<"-----Delete from Beggining-----\n";
        if(START == nullptr)
        {
            cout<<"Link List is Empty...\n\n";
            return 0;
        }
        else if (START->NEXT == nullptr)
        {
            TEMP = START;
            END = START = nullptr;
        }else
        {
            TEMP = START;
            START = TEMP->NEXT;
        }

        CountNode--;
        delete(TEMP);
        cout<<"Node Deleted Successfully...\n";
    }
}
```

```
else if (x == 2)
{
    cout<<"-----Delete from End-----\n";
    if(START == nullptr)
    {
        cout<<"Link List is Empty...\n\n";
        return 0;
    }
    else if (START->NEXT == nullptr)
    {
        TEMP = START;
        END = START = nullptr;
    }
    else
    {
        TEMP = START ;
        while(TEMP->NEXT != END){
            TEMP = TEMP->NEXT;
        }
        NODE *TEMP1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
        delete(TEMP1);
    }
    CountNode--;
    cout<<"Node Deleted Successfully...\n";
}
else if (x == 3)
{
    cout<<"\n-----Delete from given Position-----\n";
    if (START == nullptr){
        cout<<"Link List is Empty...\n\n";
        return 0;
    }
    int pos ,i;
    cout<<"Enter Position Between 1 and "<<CountNode<<" - ";
    cin>>pos;
    if (pos == 1 && START==END){
        TEMP = START ;
        END = START = nullptr;
    }
    else{
        TEMP =START;
        for(i = 2 ; i < pos ; i++){
            TEMP = TEMP->NEXT;
```

```

    }
    if(TEMP->NEXT == END){
        NODE *TEMP1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
    }else{
        NODE *TEMP1 = TEMP->NEXT;
        TEMP->NEXT = TEMP1->NEXT;
        delete(TEMP1);
    }
}
CountNode--;
cout<<"\nNode Deleted Successfully...\n";
}
char c = ContinueOperation();
if(c == '1'){
    DeleteNode(x);
}else if(c == '2'){
    DeleteOperation();
}else if(c == '3'){
    MainMenu();
}
}

//Continue Menu
int ContinueOperation()
{
    cout<<"\nDo you want to -- \n";
    cout<<"1.Continue the Same\n";
    cout<<"2.Go to Previous Menu\n";
    cout<<"3.Go to Main Menu\n";
    cout<<"4.Exit\n";
    char c = getch();
    if(c == '4'){
        thankyouLine();
        exit(0);
    }else {
        return(c);
    }
}

//Count Node Module
void countnode()
{

```



```
system("cls");
cout<<"\n=====Count Node Module=====\\n\\n";
cout<<"Total Number of Nodes -- "<<CountNode<<" Available Nodes\\n\\n";
cout<<"Do you want to -- \\n";
cout<<"1.Continue the Same\\n";
cout<<"2.Go to Main Menu\\n";
cout<<"3.Exit\\n";
char c = getch();
if(c == '1'){
    countnode();
}else if(c == '2'){
    MainMenu();
}else if(c == '3'){
    exit(0);
}
}
//MAIN MENU
void MainMenu()
{
    system("cls");
    cout<<"\n=====Singly LinkList Operations=====\\n";
    cout<<"1.Insertion Operation\\n";
    cout<<"2.Traversal Operation\\n";
    cout<<"3.Search Operation\\n";
    cout<<"4.Delete Operation\\n";
    cout<<"5.Count Node\\n";
    cout<<"6.Go To Main Link List Menu\\n\\n";
    cout<<"Choose one of Them -- ";
    char c = getch();
    if(c == '1'){
        InsertOperation();
    }else if (c == '2'){
        Traversal();
    }else if (c == '3'){
        SearchOperation();
    }else if (c == '4'){
        DeleteOperation();
    }else if (c == '5'){
        countnode();
    }else if (c == '6'){
        cout << "\\n\\nThank you for using my Singly Link List Operations...\\n\\n";
        return;
    }
}
```

```
void thankyouLine()
{
    cout<<"\n\nThank you for using my Application...\n\n";
}

};
```

SINGLY LINKED LIST OUTPUT UI

Singly Main Menu

```
=====Singly LinkList Operations=====
1.Insertion Operation
2.Traversal Operation
3.Search Operation
4.Delete Operation
5.Count Node
6.Go To Main Link List Menu

Choose one of Them --
```

► Insertion Menu

```
=====Insertion Opertaion=====
1.Insert at Begginig0
2.Insert at End
3.Insert at given Position
4.Go to Main Menu
5.Exit
Choose one of Them --
```

1. Insert at Beginning

A new node is created and linked at the head. The previous head becomes the second node.

```
=====Insertion Opertaion=====
1.Insert at Begginig
2.Insert at End
3.Insert at given Position
4.Go to Main Menu
5.Exit
Choose one of Them -- 1

-----Insertion at Begginig-----
Enter TID :-TWEET1001
Enter User Name :- Tejasvi
Enter Tweet :- MY name is Tejasvi

Node Inserted at Beginning Successfully...

Do you want to --
1.Continue the Same
2.Go to Previous Menu
3.Go to Main Menu
4.Exit
```

2. Insert at End

Traverse to the last node, set its next pointer to the new node.

```
-----Insertion at End-----  
Enter TID :-TWEET1002  
Enter User Name :- Aarav  
Enter Tweet :- Hello World  
  
Node Inserted at End Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

3. Insert at Position

Traverse to position-1, splice the new node between it and the next node.

```
-----Insertion at given Position-----  
Total Number of Nodes Are -- 0  
Enter Position Between 0 and 0- 0  
  
Enter TID :-TWEET1001  
Enter User Name :- Tejasvi  
Enter Tweet :- How are you?  
  
Node Inserted at Position Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

► Deletion Menu

```
=====Deletion Opertaion=====  
1.Delete from Beggining  
2.Delete from End  
3.delete from a given Position  
4.Go to Main Menu
```

```
5.Exit  
Choose one of Them --
```

1. Delete from Beginning

Move head to head->next, free the old head node.

```
-----Delete from Begginig-----  
  
Node Deleted Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

2. Delete from End

Walk to second-last node, set its next to NULL, free last node.

```
-----Delete from End-----  
  
Node Deleted Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

3. Delete from Position

Walk to position-1, bypass the target node, free it.

```
-----Delete from given Position-----  
Enter Position Between 1 and 1- 1  
  
Node Deleted Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

► Searching Menu

```
=====Search Module=====
1.Search by Tweet ID
2.Search by User Name
3.Main Menu
4.Exit
Choose one of Them --
```

1. Search Operation

Linear search by Tweet ID or User Name; print matching node.

```
=====Search Module=====
1.Search by Tweet ID
2.Search by User Name
3.Main Menu
4.Exit
Choose one of Them -- 1

-----Search Tweet by Tweet ID-----
Enter Tweet ID to find the Tweet -- 1001

[ TWEET1001 , Tejasvi , How are you? ]

Do you want to --
1.Continue the Same
2.Go to Main Menu
3.Exit
```

► Traversal Menu

```
-----Link List-----
Head -> [ TWEET1001 , Tejasvi , How are you? ] -> Null

Do you want to --
1.Continue the Same
2.Go to Main Menu
3.Exit
```

1. Traversal

Walk from head to NULL printing each node.

```
-----Link List-----  
Head -> [ TWEET1001 , Tejasvi , How are you? ] -> Null
```

```
Do you want to --  
1.Continue the Same  
2.Go to Main Menu  
3.Exit
```

► Count Menu

```
=====Count Node Module=====
```

Total Number of Nodes -- 1 Available Nodes

```
Do you want to --  
1.Continue the Same  
2.Go to Main Menu  
3.Exit
```

7.3 DoublyLinkedList.hpp

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class DNODE
{
public:
    DNODE *PREV ;
    string ID;
    string UNAME;
    string TWEET;
    DNODE *NEXT ;
    DNODE(string X , string Y , string Z)
    {
        PREV = nullptr;
        ID = X;
        UNAME = Y;
        TWEET = Z;
        NEXT = nullptr;
    }
};
class Doublylinklist
{
public:
    string TID;
    string UNAME;
    string UTWEET;
    int CountNode;
    int AutoTID;
    DNODE *START , *END ;
    Doublylinklist()
    {
        AutoTID=1001;
        CountNode=0;
        START=nullptr;
        END=nullptr;
    }

    //Get Data
    void getdata()
    {
        TID = "TWEET" + to_string(AutoTID);
```



```

    cout<<"Enter TID :-"<<TID<<endl;
    cout<<"Enter User Name :- ";
    getline(cin,UNAME);
    cout<<"Enter Tweet :- ";
    getline(cin,UTWEET);
}

//Insertion Operation
void InsertOperation()
{
    system("cls");
    cout<<"=====Insertion Opertaion=====\\n";
    cout<<"1.Insert at Beggining\\n";
    cout<<"2.Insert at End\\n";
    cout<<"3.Insert at given Position\\n";
    cout<<"4.Go to Main Menu\\n";
    cout<<"5.Exit\\n";
    char c = getch();
    if(c == '1'){
        InsertNode(1);
    }else if (c == '2'){
        InsertNode(2);
    }else if (c == '3'){
        InsertNode(3);
    }else if (c == '4'){
        MainMenu();
    }else if (c == '5'){
        thankyouLine();
        exit(0);
    }
}

//Insert Module
int InsertNode(int x)
{
    system("cls");
    if(x == 1){
        cout<<"-----Insertion at Beggining-----\\n";
        getdata();
        DNODE *TEMP = new DNODE(TID,UNAME,UTWEET);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory\\n";
            return 0;
        }
        if(START == nullptr){

```

```
        END = START = TEMP;
    }else {
        TEMP->NEXT = START;
        START->PREV = TEMP;
        START = TEMP;
    }
    CountNode++;
    AutoTID++;
}
else if (x == 2){
    cout<<"-----Insertion at End-----\n";
    getdata();
    DNODE *TEMP = new DNODE(TID,UNAME,UTWEET);
    if (TEMP == nullptr){
        cout<<"Insufficient Memory\n";
        return 0;
    }
    if(START == nullptr){
        END = START = TEMP;
    }else {
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    CountNode++;
    AutoTID++;
}
else if (x == 3){
    cout<<"-----Insertion at given Position-----\n";
    cout<<"Total Number of Nodes Are -- "<<CountNode<<endl;
    int pos;
    getdata();
    cout<<"Enter Position Between 0 and "<<CountNode<<"- ";
    cin>>pos;
    DNODE *TEMP = new DNODE(TID,UNAME,UTWEET);
    if (TEMP == nullptr){
        cout<<"Insufficient Memory";
        return 0;
    }
    int i ;
    DNODE *TEMP1 = START;
    for(i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if(START == nullptr){
        END = START = TEMP;
    }else {
```

```

        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT=TEMP;
        TEMP->PREV = TEMP1;//!this is the question
        TEMP->NEXT->PREV =TEMP;//!this is the question
        if (TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    CountNode++;
    AutoTID++;
}
char c = ContinueOperation();
if(c == '1'){
    InsertNode(x);
}else if(c == '2'){
    InsertOperation();
}else if(c == '3'){
    MainMenu();
}
}

//Traversal Module
void TraversalOperation()
{
    system("cls");
    cout<<"=====Traversal Module=====\\n\\n";
    cout<<"1.Forward Traversal \\n";
    cout<<"2.Backward Traversal \\n";
    cout<<"3.Main Menu \\n";
    cout<<"4.Exit\\n\\n";
    char c = getch();
    if(c == '1'){
        Traversal(1);
    }else if(c == '2'){
        Traversal(2);
    }else if(c == '3'){
        MainMenu();
    }else if(c == '4'){
        exit(0);
    }
}

//Show Module
int Traversal(int x)

```

```

{
    system("cls");
    if(x == 1){
        cout<<"\n\n-----Link List-----\n\n";
        cout<<"NULL <- ";
        DNODE *TEMP = START;
        if ( START == nullptr){
            cout<<"NULL \n\nLinklist is Empty";
            return 0;
        }
        else {
            while(TEMP != nullptr){
                cout<<"[ "<<TEMP->ID<<" , ";
                cout<<TEMP->UNAME<<" , ";
                cout<<TEMP->TWEET<<" ] -> ";
                TEMP=TEMP->NEXT;
            }
            cout<<"Null";
        }
    }
    else if(x == 2){
        cout<<"\n\n-----Link List-----\n\n";
        cout<<"NULL <- ";
        DNODE *TEMP = END;
        if ( START == nullptr){
            cout<<"NULL \n\nLinklist is Empty";
        }
        else {
            while(TEMP != nullptr){
                cout<<"[ "<<TEMP->ID<<" , ";
                cout<<TEMP->UNAME<<" , ";
                cout<<TEMP->TWEET<<" ] -> ";
                TEMP=TEMP->PREV;
            }
            cout<<"Null";
        }
    }
}
cout<<"\n\nDo you want to -- \n";
cout<<"1.Continue the Same\n";
cout<<"2.Go to Main Menu\n";
cout<<"3.Exit";
char c = getch();
if(c == '1'){
    Traversal(x);
}
else if(c == '2'){
    MainMenu();
}

```

```
}else if(c == '3'){
    thankyouLine();
    exit(0);
}
}

//Search Operation
void SearchOperation()
{
    system("cls");
    cout<<"=====Search Module=====\\n\\n";
    cout<<"1.Search by Tweet ID \\n";
    cout<<"2.Search by User Name \\n";
    cout<<"3.Main Menu \\n";
    cout<<"4.Exit\\n\\n";
    char c = getch();
    if(c == '1'){
        searchNode(1);
    }else if(c == '2'){
        searchNode(2);
    }else if(c == '3'){
        MainMenu();
    }else if(c == '4'){
        exit(0);
    }
}

//Search Module
int searchNode(int x)
{
    system("cls");
    if (x == 1){
        cout<<"-----Search Tweet by Tweet ID-----\\n\\n";
        DNODE *TEMP = START;
        if (START == nullptr){
            cout<<"Head -> Null \\n\\nLink List is Empty...";
            return 0;
        }else{
            int counter=0;
            cout<<"Enter Tweet ID to find the Tweet -- ";
            string TweetID;
            getline(cin,TweetID);
            while (TEMP != nullptr){
                if(TEMP->ID == ("TWEET" + TweetID)){
                    cout<<"\\n[ "<<TEMP->ID<<" , ";
                }
            }
        }
    }
}
```

```

        cout<<TEMP->UNAME<<" , ";
        cout<<TEMP->TWEET<<" ]\n";
        counter++;
    }
    TEMP = TEMP->NEXT;
}
if(counter == 0){
    cout<<"\nNo Tweet Found\n\n";
}
}
}
else if(x == 2){
    cout<<"-----Search Tweet by User Name-----\n\n";

    DNODE *TEMP = START;
    if (START == nullptr){
        cout<<"Head -> Null \n\nLink List is Empty...";
        return 0;
    }else{
        int counter=0 ;
        cout<<"Enter user Name to find the Tweet -- ";
        string uname;
        getline(cin,uname);
        cout<<endl<<"Head -> ";
        while (TEMP != nullptr){
            if(TEMP->UNAME == uname){
                cout<<"\n[ "<<TEMP->ID<<" , ";
                cout<<TEMP->UNAME<<" , ";
                cout<<TEMP->TWEET<<" ]\n";
                counter++;
            }
            TEMP = TEMP->NEXT;
        }
        if(counter == 0){
            cout<<"\nNo Tweet Found\n\n";
        }
    }
}
cout<<"Do you want to -- \n";
cout<<"1.Continue the Same\n";
cout<<"2.Go to Main Menu\n";
cout<<"3.Exit\n";
char c = getch();
if(c == '1'){
    searchNode(x);
}

```

```

    }else if(c == '2'){
        MainMenu();
    }else if(c == '3'){
        thankyouLine();
        exit(0);
    }
}

//Delete Operation
void DeleteOperation()
{
    system("cls");
    cout<<"=====Deletion Opertaion=====\\n\\n";
    cout<<"1.Delete from Beggining\\n";
    cout<<"2.Delete from End\\n";
    cout<<"3.delete from a given Position\\n";
    cout<<"4.Go to Main Menu\\n";
    cout<<"5.Exit\\n";
    char c = getch();
    if(c == '1'){
        DeleteNode(1);
    }else if (c == '2'){
        DeleteNode(2);
    }else if (c == '3'){
        DeleteNode(3);
    }else if (c == '4'){
        MainMenu();
    }else if (c == '5'){
        thankyouLine();
        exit(0);
    }
}

//Delete Module
int DeleteNode(int x)
{
    system("cls");
    DNODE *TEMP ;
    if(x == 1){
        cout<<"-----Delete from Beggining-----\\n";
        if(START == nullptr)
        {
            cout<<"Link List is Empty...\\n\\n";
            return 0;
        }
        else if (START->NEXT == nullptr)

```

```
{
    TEMP = START;
    END = START = nullptr;
}else
{
    TEMP = START;
    START = TEMP->NEXT;
    START->PREV = nullptr;
}

CountNode--;
delete(TEMP);
cout<<"\nNode Deleted Successfully...\n";

}
else if (x == 2)
{
    cout<<"-----Delete from End-----\n";
    if(START == nullptr)
    {
        cout<<"Link List is Empty...\n\n";
        return 0;
    }
    else if (START->NEXT == nullptr)
    {
        TEMP = START;
        END = START = nullptr;
    }
    else
    {
        TEMP = END;
        END = END->PREV;
        END->NEXT=nullptr;
        /*!
        TEMP=END;
        TEMP->PREV->NEXT=NULLPTR;
        END=TEMP->PREV;
        */
        delete(TEMP);
    }
    CountNode--;
    cout<<"\nNode Deleted Successfully...";
}
else if (x == 3)
{
```



```

        cout<<"\n\n-----Delete from given Position-----\n";
        if (START == nullptr){
            cout<<"Link List is Empty...\n\n";
            return 0;
        }
        int pos ,i;
        cout<<"Enter Position Between 1 and "<<CountNode<<"- ";
        cin>>pos;
        if (pos == 1 && START==END){
            TEMP = START ;
            END = START = nullptr;
        }
        else{
            TEMP =START;
            for(i = 1 ; i < pos ; i++){
                TEMP = TEMP->NEXT;
            }
            if(TEMP == END){
                TEMP->PREV->NEXT = nullptr;
                END = TEMP->PREV;
                delete(TEMP);
            }else{
                TEMP->PREV->NEXT = TEMP->NEXT;
                TEMP->NEXT->PREV = TEMP->PREV;
                delete(TEMP);
            }
        }
        CountNode--;
        cout<<"Node Deleted Successfully...";
    }
    char c = ContinueOperation();
    if(c == '1'){
        DeleteNode(x);
    }else if(c == '2'){
        InsertOperation();
    }else if(c == '3'){
        MainMenu();
    }
}

//Continue Module
int ContinueOperation()
{
    cout<<"Do you want to -- \n";

```

```
cout<<"1.Continue the Same\n";
cout<<"2.Go to Previous Menu\n";
cout<<"3.Go to Main Menu\n";
cout<<"4.Exit";
char c = getch();
if(c == '4'){
    thankyouLine();
    exit(0);
}else {
    return(c);
}
}

//Count Node Module
void countnode()
{
    system("cls");
    cout<<"=====Count Node Module=====";
    cout<<"\n\nTotal Number of Nodes -- "<<CountNode<<"Available Nodes\n\n";
    cout<<"Do you want to -- \n";
    cout<<"1.Continue the Same\n";
    cout<<"2.Go to Main Menu\n";
    cout<<"3.Exit";
    char c = getch();
    if(c == '1'){
        countnode();
    }else if(c == '2'){
        MainMenu();
    }else if(c == '3'){
        exit(0);
    }
}

//Main Menu
void MainMenu()
{
    system("cls");
    cout<<"=====Doubly LinkList Operations=====\\n";
    cout<<"1.Insertion Operation\n";
    cout<<"2.Traversal Operation\n";
    cout<<"3.Search Operation\n";
    cout<<"4.Delete Operation\n";
    cout<<"5.Count Node\n";
    cout<<"6.Go To Main Link List Menu\n\n";
    char c = getch();
    if(c == '1'){
```

```
        InsertOperation();
    }else if (c == '2'){
        TraversalOperation();
    }else if (c == '3'){
        SearchOperation();
    }else if (c == '4'){
        DeleteOperation();
    }else if (c == '5'){
        countnode();
    }else if (c == '6'){
        cout << "\n\nThank you for using my Doubly Link List Operations...\n\n";
        return;
    }
}

void thankyouLine()
{
    cout<<"\n\nThank you for using my Application...\n\n";
}

};
```

DOUBLY LINKED LIST OUTPUT

► Doubly Main Menu

```
=====Doubly Linklist Operations=====
1.Insertion Operation
2.Traversal Operation
3.Search Operation
4.Delete Operation
5.Count Node
6.Go To Main Link List Menu

Choose one of Them --
```

► Insertion Menu

```
=====Insertion Opertaion=====
1.Insert at Beggining
2.Insert at End
3.Insert at given Position
4.Go to Main Menu
5.Exit
Choose one of Them --
```

1. Insert at Beginning

New node becomes head; old head's prev points to it.

```
=====Insertion Opertaion=====
1.Insert at Beggining
2.Insert at End
3.Insert at given Position
4.Go to Main Menu
5.Exit
Choose one of Them -- 1

-----Insertion at Beggining-----
Enter TID :-TWEET1001
Enter User Name :- Tejasvi
Enter Tweet :- MY name is Tejasvi

Node Inserted at Beginning Successfully...

Do you want to --
1.Continue the Same
2.Go to Previous Menu
3.Go to Main Menu
4.Exit
```

2. Insert at End

Walk to last; set last->next = new, new->prev = last.

```
-----Insertion at End-----  
Enter TID :-TWEET1002  
Enter User Name :- Aarav  
Enter Tweet :- Doubly node added  
  
Node Inserted at End Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

3. Insert at Position

Re-wire four pointers: prev<->new and new<->next.

```
-----Insertion at given Position-----  
Total Number of Nodes Are -- 1  
Enter Position Between 1 and 2- 2  
  
Enter TID :-TWEET1003  
Enter User Name :- Riya  
Enter Tweet :- Inserted at pos 2  
  
Node Inserted at Position Successfully...  
  
Do you want to --  
1.Continue the Same  
2.Go to Previous Menu  
3.Go to Main Menu  
4.Exit
```

► Deletion Menu

```
=====Deletion Opertaion=====  
1.Delete from Beggining  
2.Delete from End  
3.delete from a given Position  
4.Go to Main Menu  
5.Exit  
Choose one of Them --
```

1. Delete from Beginning

head = head->next; head->prev = NULL; free old head.

```
-----Delete from Beggining-----
```

```
Node Deleted Successfully...
```

```
Do you want to --
```

- 1.Continue the Same
- 2.Go to Previous Menu
- 3.Go to Main Menu
- 4.Exit

2. Delete from End

Walk to last; last->prev->next = NULL; free last.

```
-----Delete from End-----
```

```
Node Deleted Successfully...
```

```
Do you want to --
```

- 1.Continue the Same
- 2.Go to Previous Menu
- 3.Go to Main Menu
- 4.Exit

3. Delete from Position

Re-wire prev->next and next->prev around the target, free target.

```
-----Delete from given Position-----
```

```
Enter Position Between 1 and 2- 2
```

```
Node Deleted Successfully...
```

```
Do you want to --
```

- 1.Continue the Same
- 2.Go to Previous Menu
- 3.Go to Main Menu
- 4.Exit

► Searching Menu

```
=====Search Module=====
```

- 1.Search by Tweet ID
- 2.Search by User Name
- 3.Main Menu
- 4.Exit

```
Choose one of Them --
```

1. Search Operation

Linear scan by TID or User Name; both directions available.

```
=====Search Module=====
1.Search by Tweet ID
2.Search by User Name
3.Main Menu
4.Exit
Choose one of Them -- 2

-----Search Tweet by User Name-----
Enter user Name to find the Tweet -- Tejasvi

Head ->
[ TWEET1001 , Tejasvi , MY name is Tejasvi ]

Do you want to --
1.Continue the Same
2.Go to Main Menu
3.Exit
```

► Traversal Menu

```
=====Traversal Module=====
1.Forward Traversal
2.Backward Traversal
3.Main Menu
4.Exit
Choose one of Them --
```

1. Forward & Backward Traversal

Forward: head -> next ... NULL. Backward: tail -> prev ... NULL.

```
-----Link List-----

NULL <- [ TWEET1001 , Tejasvi , MY name is Tejasvi ] -> Null

-----Reverse Link List-----

NULL <- [ TWEET1001 , Tejasvi , MY name is Tejasvi ] -> Null

Do you want to --
1.Continue the Same
2.Go to Main Menu
3.Exit
```

► Count Menu

```
=====Count Node Module=====
```

```
Total Number of Nodes -- 1 Available Nodes
```

```
Do you want to --
```

```
1.Continue the Same
```

```
2.Go to Main Menu
```

```
3.Exit
```




Scan to see github repository and source code

Thank You

Tejasvi Maheshwari
BCA 2nd Semester

Dezyne École College